claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a4cad663cc1d05fd9.jsonl`
- SHA-256: `7756b688c8f7cad7461e5be5a0725d856c613974d053389f313b447f94e3c0d5`
- Source modified: `2026-06-10T06:12:09+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T06:09:28.017Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("collector") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"Silent destructive replace of golden labels: DELETE-then-INSERT + video_id UPSERT with no warning, backup, or history","severity":"high","area":"db layer / import-local","file":"C:/Users/avidu/Projects/sports-data-collector/src/core/db.py:210-237","evidence":"Every insert_*_rallies first runs `DELETE FROM <table> WHERE video_id = ?` (db.py:217, 245, 272, 301, 329). insert_video is an unconditional UPSERT (db.py:178-206). Verified empirically on a temp DB: 40 existing golden rallies -> re-import of a 1-row file -> 1 rally, no warning; re-registering video_id 'v1' as a different sport/match silently overwrote the metadata row while leaving the old sport's child rows orphaned. video_id is auto-derived from the video filename when --video-id is omitted (curate.py:68-69: basename.split('.')[0].lower().replace(' ','_')), so two files named 'match.mp4' collide.","why_it_matters":"For a system-of-record there is no protection against accidentally clobbering vendor-QA'd golden labels with a partial/wrong file: no 'video_id already exists, N rallies will be replaced' prompt, no audit trail, no soft-delete. One mistyped or defaulted video_id silently rewrites corpus ground truth."}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:09:34.544Z)

```
160                 # do it explicitly and idempotently (skip if the column is already there).
161                 for tbl in ("badminton_rallies", "pickleball_rallies",
"tabletennis_rallies"):
162                     existing = [r[1] for r in cursor.execute(f"PRAGMA
table_info({tbl})").fetchall()]
163                     if "last_shot_outcome" not in existing:
164                         cursor.execute(f"ALTER TABLE {tbl} ADD COLUMN last_shot_outcome
TEXT")
165
166                 conn.commit()
167
168         def insert_video(self, video: VideoMetadata) -> bool:
169             """Insert a video entry, or update it in place if it already exists.
170
171             Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
172             existing row, which (with foreign keys enabled) would cascade-delete the
173             video's child annotations. UPSERT mutates the row in place and leaves
174             children untouched.
175             """
176             with self._get_connection() as conn:
177                 cursor = conn.cursor()
178                 cursor.execute("""
179                 INSERT INTO videos (
180                     video_id, sport, video_url, local_path, license_type,
181                     license_url, is_commercial, status, match_name, fps, resolution
182                 ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
183                 ON CONFLICT(video_id) DO UPDATE SET
184                     sport=excluded.sport,
185                     video_url=excluded.video_url,
186                     local_path=excluded.local_path,
187                     license_type=excluded.license_type,
188                     license_url=excluded.license_url,
189                     is_commercial=excluded.is_commercial,
190                     status=excluded.status,
191                     match_name=excluded.match_name,
192                     fps=excluded.fps,
193                     resolution=excluded.resolution
194                 """, (
195                     video.video_id,
196                     video.sport.value,
197                     video.video_url,
198                     video.local_path,
199                     video.license_type.value,
200                     video.license_url,
201                     1 if video.is_commercial else 0,
202                     video.status.value,
203                     video.match_name,
204                     video.fps,
205                     video.resolution
206                 ))
207                 conn.commit()
208                 return True
209
210         def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
211             if not rallies:
212                 return 0
213             with self._get_connection() as conn:
214                 cursor = conn.cursor()
215                 # First clear existing rallies for this video_id to avoid key conflicts on
updates
216                 video_id = rallies[0].video_id
217                 cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
218
219                 for rally in rallies:
```

```
                    cursor.execute("""
                    INSERT INTO badminton_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason,
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)

        def insert_tennis_points(self, points: List[TennisPoint]) -> int:
            if not points:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = points[0].video_id
                cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))

                for pt in points:
                    cursor.execute("""
                    INSERT INTO tennis_points (
                        video_id, point_number, start_time, end_time,
                        set_score, game_score, server, ending_type
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        pt.video_id,
                        pt.point_number,
                        pt.start_time,
                        pt.end_time,
                        pt.set_score,
                        pt.game_score,
                        pt.server,
                        pt.ending_type
                    ))
                conn.commit()
                return len(points)

        def insert_cricket_deliveries(self, deliveries: List[CricketDelivery]) -> int:
            if not deliveries:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = deliveries[0].video_id
                cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))

                for deliv in deliveries:
                    cursor.execute("""
                    INSERT INTO cricket_deliveries (
                        video_id, over_num, ball_num, start_time, end_time,
                        batsman, bowler, runs, wicket, extras
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        deliv.video_id,
                        deliv.over,
```

```
                        deliv.ball,
                        deliv.start_time,
                        deliv.end_time,
                        deliv.batsman,
                        deliv.bowler,
                        deliv.runs,
                        1 if deliv.wicket else 0,
                        deliv.extras
                    ))
                conn.commit()
                return len(deliveries)

        def insert_pickleball_rallies(self, rallies: List[PickleballRally]) -> int:
            if not rallies:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
(video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO pickleball_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason,
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)

        def insert_tabletennis_rallies(self, rallies: List[TableTennisRally]) -> int:
            if not rallies:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO tabletennis_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason,
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
```

```
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)
```

3. user (2026-06-10T06:09:35.406Z)

C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py

4. user (2026-06-10T06:09:46.054Z)

```
1       import os
2       import sys
3       import time
4       import json
5       import csv
6       import argparse
7       from typing import List, Dict, Any, Tuple
8       from tabulate import tabulate
9
10      from src.core.config import load_config
11      from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12      from src.core.db import SportsDatabase
13      from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14      from src.core.schemas.badminton import BadmintonRally
15      from src.core.schemas.tennis import TennisPoint
16      from src.core.schemas.cricket import CricketDelivery
17      from src.core.schemas.pickleball import PickleballRally
18      from src.core.schemas.tabletennis import TableTennisRally
19      from src.core.validator import DatasetValidator
20      from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23      class SportsDataCLI:
24          def __init__(self, config_path: str = "config/config.yaml"):
25              self.config_path = config_path
26              self.config = self._load_config()
27
28              # Resolve DB path
29              db_path = self.config.get("database_path", "data/sports_metadata.db")
30              self.db = SportsDatabase(db_path)
31              self.validator = DatasetValidator(config_path)
32
33          def _load_config(self) -> Dict[str, Any]:
34              if not os.path.exists(self.config_path):
35                  print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36              return load_config(self.config_path)
37
38          def status(self):
39              """Displays status summary of all videos in the database."""
40              # Query total count by status via the public DB read API.
41              rows = self.db.get_status_counts()  # list of (sport, status, cnt)
42
43              if not rows:
44                  print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45                  return
46
47              data = [[sport, status, cnt] for sport, status, cnt in rows]
48              print("\n=== Dataset Ingestion Status ===")
49              print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50              print()
```

```
51
52          def import_local(self, args):
53              """Manually registers a local video and annotation file into the database."""
54              sport_str = args.sport.lower()
55              if sport_str not in self.config.get("supported_sports", ["badminton"]):
56                  print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57                  sys.exit(1)
58
59              # Validate video path
60              if not os.path.exists(args.video_path):
61                  print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")
62
63              # Validate annotation file
64              if not os.path.exists(args.annotations_path):
65                  print(f"Error: Annotations file '{args.annotations_path}' not found.")
66                  sys.exit(1)
67
68              match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
69              video_id = args.video_id or match_name.lower().replace(" ", "_")
70
71              # Parse license
72              try:
73                  license_type = LicenseType(args.license)
74              except ValueError:
75                  print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
76                  license_type = LicenseType.UNKNOWN
77
78              is_commercial = self.validator.check_license_commercial(license_type)
79
80              # 1. Parse Annotations
81              annotations = []
82              try:
83                  annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
84              except Exception as e:
85                  print(f"Error parsing annotations: {e}")
86                  sys.exit(1)
87
88              # 2. Run Validation
89              is_valid = True
90              errors = []
91              if sport_str == "badminton":
92                  is_valid, errors = self.validator.validate_badminton_rallies(annotations)
93              elif sport_str == "tennis":
94                  is_valid, errors = self.validator.validate_tennis_points(annotations)
95              elif sport_str == "cricket":
96                  is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
97              elif sport_str == "pickleball":
98                  is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
99              elif sport_str == "tabletennis":
100                 is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
101
102             if not is_valid:
103                 print(f"Error: Annotations failed validation!")
104                 for err in errors:
105                     print(f"  - {err}")
106                 sys.exit(1)
107
108             # 3. Create Video Metadata
109             video_meta = VideoMetadata(
110                 video_id=video_id,
111                 sport=SportType(sport_str),
```

```
                video_url=None,
                local_path=args.video_path,
                license_type=license_type,
                license_url=None,
                is_commercial=is_commercial,
                status=CurationStatus.CURATED,  # Custom local imports bypass staging by default
                match_name=match_name,
                fps=args.fps,
                resolution=args.resolution
            )

            # 4. Insert into Database
            self.db.insert_video(video_meta)

            count = 0
            if sport_str == "badminton":
                count = self.db.insert_badminton_rallies(annotations)
            elif sport_str == "tennis":
                count = self.db.insert_tennis_points(annotations)
            elif sport_str == "cricket":
                count = self.db.insert_cricket_deliveries(annotations)
            elif sport_str == "pickleball":
                count = self.db.insert_pickleball_rallies(annotations)
            elif sport_str == "tabletennis":
                count = self.db.insert_tabletennis_rallies(annotations)

            print(f"\nSuccessfully imported local video '{match_name}':")
            print(f"  - Video ID: {video_id}")
            print(f"  - Sport: {sport_str}")
            print(f"  - Commercial Use Allowed: {is_commercial} (License: {license_type.value})")
            print(f"  - Registered {count} annotated segments.\n")

            # Per-video observability report (next to the video). Never raises.
            from src.reporting import emit_import_report
            src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or "unknown"
            report_paths = emit_import_report(
                video_meta_obj=video_meta, annotations=annotations, is_valid=True,
                errors=[], count=count, sport=sport_str, source_format=src_fmt,
                fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
            )
            if report_paths:
                print(f"  - Observability report: {report_paths.get('technical_md')}")
                print(f"  - Customer summary    : {report_paths.get('customer_md')}\n")

        def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) -> List[Any]:
            """Parses simple JSON or CSV annotations files."""
            parsed_items = []
            ext = os.path.splitext(filepath)[1].lower()

            if ext == '.json':
                with open(filepath, 'r', encoding='utf-8') as f:
                    data = json.load(f)

                if not isinstance(data, list):
                    raise ValueError("JSON file must contain a list of objects.")

                # Uses the same safe casts as the CSV path: a missing/blank required
                # time raises a clear ValueError (caught by the caller) instead of a
                # bare KeyError, and optional ints degrade to sensible defaults.
                for idx, obj in enumerate(data):
                    if sport == "badminton":
```

```
                        parsed_items.append(BadmintonRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
                    elif sport == "tennis":
                        parsed_items.append(TennisPoint(
                            video_id=video_id,
                            point_number=_safe_int(obj.get("point_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            set_score=obj.get("set_score"),
                            game_score=obj.get("game_score"),
                            server=obj.get("server"),
                            ending_type=obj.get("ending_type")
                        ))
                    elif sport == "cricket":
                        parsed_items.append(CricketDelivery(
                            video_id=video_id,
                            over=_safe_int(obj.get("over"), 0),
                            ball=_safe_int(obj.get("ball"), 0),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            batsman=obj.get("batsman"),
                            bowler=obj.get("bowler"),
                            runs=_safe_int(obj.get("runs"), None),
                            wicket=bool(obj.get("wicket", False)),
                            extras=_safe_int(obj.get("extras"), 0)
                        ))
                    elif sport == "pickleball":
                        parsed_items.append(PickleballRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
                    elif sport == "tabletennis":
                        parsed_items.append(TableTennisRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
            elif ext == '.csv':
                with open(filepath, 'r', encoding='utf-8') as f:
                    reader = csv.DictReader(f)
                    # Normalize headers
                    reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []
```

```
237
238                     for idx, row in enumerate(reader):
239                         if sport == "badminton":
240                             parsed_items.append(BadmintonRally(
241                                 video_id=video_id,
242                                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
243                                 start_time=_safe_float(row.get("start_time")),
244                                 end_time=_safe_float(row.get("end_time")),
245                                 score_before=row.get("score_before"),
246                                 score_after=row.get("score_after"),
247                                 shots_count=_safe_int(row.get("shots_count"), None),
248                                 ending_reason=row.get("ending_reason"),
249                                 last_shot_outcome=row.get("last_shot_outcome")
250                             ))
251                         elif sport == "tennis":
252                             parsed_items.append(TennisPoint(
253                                 video_id=video_id,
254                                 point_number=_safe_int(row.get("point_number"), idx + 1),
255                                 start_time=_safe_float(row.get("start_time")),
256                                 end_time=_safe_float(row.get("end_time")),
257                                 set_score=row.get("set_score"),
258                                 game_score=row.get("game_score"),
259                                 server=row.get("server"),
260                                 ending_type=row.get("ending_type")
261                             ))
262                         elif sport == "cricket":
263                             parsed_items.append(CricketDelivery(
264                                 video_id=video_id,
265                                 over=_safe_int(row.get("over"), 0),
266                                 ball=_safe_int(row.get("ball"), 0),
267                                 start_time=_safe_float(row.get("start_time")),
268                                 end_time=_safe_float(row.get("end_time")),
269                                 batsman=row.get("batsman"),
270                                 bowler=row.get("bowler"),
271                                 runs=_safe_int(row.get("runs"), None),
272                                 wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
273                                 extras=_safe_int(row.get("extras"), 0)
274                             ))
275                         elif sport == "pickleball":
276                             parsed_items.append(PickleballRally(
277                                 video_id=video_id,
278                                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
279                                 start_time=_safe_float(row.get("start_time")),
280                                 end_time=_safe_float(row.get("end_time")),
281                                 score_before=row.get("score_before"),
282                                 score_after=row.get("score_after"),
283                                 shots_count=_safe_int(row.get("shots_count"), None),
284                                 ending_reason=row.get("ending_reason"),
285                                 last_shot_outcome=row.get("last_shot_outcome")
286                             ))
287                         elif sport == "tabletennis":
288                             parsed_items.append(TableTennisRally(
289                                 video_id=video_id,
290                                 rally_number=_safe_int(row.get("rally_number"), idx + 1),
291                                 start_time=_safe_float(row.get("start_time")),
292                                 end_time=_safe_float(row.get("end_time")),
293                                 score_before=row.get("score_before"),
294                                 score_after=row.get("score_after"),
295                                 shots_count=_safe_int(row.get("shots_count"), None),
296                                 ending_reason=row.get("ending_reason"),
297                                 last_shot_outcome=row.get("last_shot_outcome")
298                             ))
299             else:
300                 raise ValueError("Unsupported annotations file extension. Use .json or
```

```
.csv.")

            return parsed_items

        def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
            """Download one video at best resolution, retrying transient failures.

            A below-min-height result almost always means YouTube rate-limited the
            request and served a low fallback (not that the video lacks HD), so we
            retry with a growing backoff. Returns (DownloadResult, None) on success,
            or (None, failure_reason) once retries are exhausted. A below-min file is
            discarded each attempt so it never lands in the DB.
            """
            attempts = max(1, args.retries + 1)
            reason = "unknown error"
            for i in range(attempts):
                result = download_best_video(
                    url, dest_path,
                    max_height=max_height, max_size_mb=max_size_mb,
                    cookies_from_browser=args.cookies_from_browser,
                    cookies_file=args.cookies,
                    player_client=args.player_client,
                )
                if result.success and (result.height is None or result.height >=
args.min_height):
                    return result, None

                if not result.success:
                    reason = result.error or "unknown error"
                else:
                    reason = (f"got {result.resolution_label} (< min-height
{args.min_height}p) ? "
                              f"YouTube served a low format (rate-limited?)")
                    try:
                        os.remove(result.path)
                    except OSError:
                        pass

                if i < attempts - 1:
                    backoff = args.retry_backoff * (i + 1)
                    print(f"  attempt {i+1}/{attempts} failed: {reason}\n  retrying in
{backoff:.0f}s...")
                    time.sleep(backoff)
            return None, reason

        def fetch(self, args):
            """(Re)download videos at best available resolution, updating the DB in place.

            This is the in-place upgrade path: rows, rally annotations, and curation
            status are all preserved ? only the video file plus its `local_path`,
            `resolution`, and `fps` are refreshed. By default it skips videos that
            already have a local file at or above `--min-height`, so re-runs are cheap
            and resumable; `--force` re-fetches everything.
            """
            storage = self.config.get("video_storage", {})
            local_dir = storage.get("local_dir", "./data/videos")
            max_download_size_mb = storage.get("max_download_size_mb")
            max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
            os.makedirs(local_dir, exist_ok=True)

            commercial_only = not args.include_noncommercial
            if args.status == "all":
                statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
            else:
```

```
                statuses = [CurationStatus(args.status)]

            supported = self.config.get("supported_sports", ["badminton"])
            sport_strs = [args.sport] if args.sport else supported

            # Collect target videos.
            videos = []
            for sport_str in sport_strs:
                try:
                    sport = SportType(sport_str.lower())
                except ValueError:
                    print(f"Warning: skipping unknown sport '{sport_str}'.")
                    continue
                for status in statuses:
                    videos.extend(self.db.get_videos_by_status(status, sport))
            if args.video_id:
                videos = [v for v in videos if v.video_id == args.video_id]

            if not videos:
                print("\nNo matching videos to fetch.\n")
                return

            results = []  # (video_id, action, detail)
            for video in videos:
                if commercial_only and not video.is_commercial:
                    results.append((video.video_id, "skip", "non-commercial"))
                    continue
                if not video.video_url or "watch?v=example" in video.video_url:
                    # Either no URL, or the parser's placeholder (the ShuttleSet catalog
                    # had no URL for this match) ? nothing to download.
                    results.append((video.video_id, "skip", "no real source URL"))
                    continue

                dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")

                # Skip if we already have a good-enough local file (unless --force).
                if not args.force and os.path.exists(dest_path):
                    _, height, _ = probe_resolution_fps(dest_path)
                    if height is not None and height >= args.min_height:
                        results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
                        continue

                print(f"Fetching {video.video_id} ({video.match_name})...")
                result, fail_reason = self._fetch_one_with_retry(
                    video.video_url, dest_path, args, max_height, max_download_size_mb)
                if fail_reason is not None:
                    results.append((video.video_id, "FAIL", fail_reason))
                    if args.sleep:
                        time.sleep(args.sleep)
                    continue

                # Update only the file-derived fields; preserve everything else (UPSERT
                # mutates the row in place, leaving rallies intact).
                video.local_path = result.path
                if result.resolution_label:
                    video.resolution = result.resolution_label
                if result.fps:
                    video.fps = result.fps
                self.db.insert_video(video)

                detail = (f"{result.resolution_label or '?'}"
                          f"{f' {result.fps:g}fps' if result.fps else ''},
{result.size_mb:.0f} MB")
                results.append((video.video_id, "fetched", detail))
```

```
                if args.sleep:
                    time.sleep(args.sleep)

            print("\n=== Fetch summary ===")
            print(tabulate([[vid, action, detail] for vid, action, detail in results],
                           headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
            fetched = sum(1 for _, a, _ in results if a == "fetched")
            failed = [r for r in results if r[1] == "FAIL"]
            print(f"\nFetched/updated : {fetched}   Already-good : {sum(1 for _,a,_ in
results if a=='have')}   "
                  f"Skipped : {sum(1 for _,a,_ in results if a=='skip')}   Failed :
{len(failed)}")
            if failed:
                print("Failures (left unchanged in DB):")
                for vid, _, detail in failed:
                    print(f"  - {vid}: {detail}")
            print()

        def export_eval_set(self, args):
            """Export curated annotations as indexer-ready eval/training sets.

            Emits, per qualifying video, a `<out_dir>/<sport>/<video_id>.csv` in the
            indexer's `start,end` ground-truth format (decimal seconds), plus a single
            `<out_dir>/manifest.json` that pairs each CSV with its video file. The
            manifest is the contract the downstream eval/training driver reads: the
            indexer keys videos by *file MD5*, not by our `video_id`, so it must run
            against the exact `local_path`/`video_url` recorded here.

            By default only CURATED + commercially-safe videos are exported, mirroring
            the license gate already enforced elsewhere in the toolchain.
            """
            out_dir = args.out_dir
            commercial_only = not args.include_noncommercial
            statuses = [CurationStatus.CURATED]
            if args.include_staging:
                statuses.append(CurationStatus.STAGING)

            supported = self.config.get("supported_sports", ["badminton"])
            sport_strs = [args.sport] if args.sport else supported

            manifest = []
            skipped_noncomm = 0
            skipped_empty = 0

            for sport_str in sport_strs:
                try:
                    sport = SportType(sport_str.lower())
                except ValueError:
                    print(f"Warning: skipping unknown sport '{sport_str}'.")
                    continue

                videos = []
                for status in statuses:
                    videos.extend(self.db.get_videos_by_status(status, sport))

                for video in videos:
                    if commercial_only and not video.is_commercial:
                        skipped_noncomm += 1
                        continue

                    intervals = self.db.get_segment_intervals(video.video_id, sport)
                    if not intervals:
                        skipped_empty += 1
                        continue

```

```
                    sport_dir = os.path.join(out_dir, sport.value)
                    os.makedirs(sport_dir, exist_ok=True)
                    csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
                    with open(csv_path, "w", newline="", encoding="utf-8") as f:
                        writer = csv.writer(f)
                        writer.writerow(["start", "end"])
                        for start, end in intervals:
                            writer.writerow([start, end])

                    manifest.append({
                        "video_id": video.video_id,
                        "sport": sport.value,
                        "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
                        "local_path": video.local_path,
                        "video_url": video.video_url,
                        "match_name": video.match_name,
                        "fps": video.fps,
                        "resolution": video.resolution,
                        "license_type": video.license_type.value,
                        "is_commercial": video.is_commercial,
                        "status": video.status.value,
                        "num_segments": len(intervals),
                    })

            os.makedirs(out_dir, exist_ok=True)
            manifest_doc = {
                "commercial_only": commercial_only,
                "statuses": [s.value for s in statuses],
                "total_videos": len(manifest),
                "total_segments": sum(m["num_segments"] for m in manifest),
                "eval_sets": manifest,
            }
            manifest_path = os.path.join(out_dir, "manifest.json")
            with open(manifest_path, "w", encoding="utf-8") as f:
                json.dump(manifest_doc, f, indent=2)

            print(f"\n=== Exported eval/training set to '{out_dir}' ===")
            if manifest:
                summary = [[m["sport"], m["video_id"], m["num_segments"],
                            "YES" if m["is_commercial"] else "NO",
                            os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
                           for m in manifest]
                print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
            print(f"\nVideos exported : {len(manifest)}")
            print(f"Total segments  : {manifest_doc['total_segments']}")
            if skipped_noncomm:
                print(f"Skipped (non-commercial) : {skipped_noncomm}  (pass --include-
noncommercial to include)")
            if skipped_empty:
                print(f"Skipped (no annotations) : {skipped_empty}")
            print(f"Manifest        : {manifest_path}\n")

        def convert_cvat(self, args):
            """Convert a vendor CVAT rally export into our canonical per-rally CSV.

            Absorbs the CVAT bounding-box delivery format (rally fields carried as box
            attributes) on the ingestion side: each rally's time is recomputed from
            its frame range at the video's true fps, and content defects (overlaps,
            zero-duration, off-vocabulary endings, coarse sampling) are written to an
            issues report rather than silently "fixed". The emitted CSV imports via
            `import-local`; any blocker issue must be resolved first, since the
            importer's validator rejects overlapping / zero-duration rallies.
            """
```

```
            from src.parsers.cvat.rally_cvat import (
                convert, write_canonical_csv, render_issue_report,
            )
            try:
                result = convert(args.input, fps=args.fps,
                                 video_path=args.video_path, step=args.step)
            except (FileNotFoundError, ValueError) as e:
                print(f"Error: {e}")
                sys.exit(1)

            write_canonical_csv(result.records, args.out)

            # Observability report next to the canonical CSV. Never raises.
            from src.reporting import emit_delivery_report
            src_name = os.path.splitext(os.path.basename(args.input))[0]
            dur = (result.meta.get("stop_frame") / result.fps) if
(result.meta.get("stop_frame") and result.fps) else None
            emit_delivery_report(
                out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
                records=result.records, issues=result.issues, fps=result.fps,
video_path=args.video_path,
                video_duration_s=dur, source_format="cvat", step=result.step,
            )

            report = render_issue_report(result, os.path.basename(args.input))
            if args.issues:
                with open(args.issues, "w", encoding="utf-8") as f:
                    f.write(report)
            print(report)
            print(f"Canonical CSV written: {args.out}  ({len(result.records)} rallies)")
            if result.n_blockers:
                print(f"\n  {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
                      f"this file until they are resolved (by the vendor or manual
review).")
            else:
                print("\n  No blockers. Next:")
                print(f"    python -m src.cli.curate import-local --sport <sport> "
                      f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")

        def validate_delivery(self, args):
            """Run the full validation suite on a rally delivery and emit a report.

            Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
            CSV (--csv). Writes, into --out-dir:
              <id>_report.md    human report (scoreboard + issues + rallies to review)
              <id>_report.json  machine summary (for CI / a bulk-ingest loop)
              <id>_review.csv   the annotatable manual-verification sheet
            Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
            """
            import json
            from src.parsers.cvat.rally_cvat import (
                convert, load_canonical_csv, detect_issues,
                build_summary, render_validation_md, write_review_csv,
            )
            vid = args.video_id or "delivery"
            fps = None
            step = None
            stop_frame = None
            if args.input:
                try:
                    result = convert(args.input, fps=args.fps, video_path=args.video_path)
                except (FileNotFoundError, ValueError) as e:
                    print(f"Error: {e}")
```

```
                    sys.exit(1)
                records, issues, fps = result.records, result.issues, result.fps
                step = result.step
                stop_frame = result.meta.get("stop_frame")
            elif args.csv:
                try:
                    records = load_canonical_csv(args.csv)
                except FileNotFoundError as e:
                    print(f"Error: {e}")
                    sys.exit(1)
                if not records:
                    print(f"Error: no usable rally rows in {args.csv}")
                    sys.exit(1)
                issues = detect_issues(records)
            else:
                print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
                sys.exit(1)

            os.makedirs(args.out_dir, exist_ok=True)
            summary = build_summary(records, issues, source=vid, fps=fps)
            md = render_validation_md(records, issues, source=vid)
            json_path = os.path.join(args.out_dir, f"{vid}_report.json")
            md_path = os.path.join(args.out_dir, f"{vid}_report.md")
            review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
            with open(json_path, "w", encoding="utf-8") as f:
                json.dump(summary, f, indent=2)
            with open(md_path, "w", encoding="utf-8") as f:
                f.write(md)
            write_review_csv(records, issues, review_path)

            # Observability report (envelope JSON + technical md + customer summary),
            # colocated with the existing artifacts. Never raises.
            from src.reporting import emit_delivery_report
            duration_s = (stop_frame / fps) if (stop_frame and fps) else None
            report_paths = emit_delivery_report(
                out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
                video_path=args.video_path, video_duration_s=duration_s,
                source_format=("cvat" if args.input else "csv"), step=step,
            )

            print(md)
            print(f"\nWrote:\n  {md_path}\n  {json_path}\n  {review_path}")
            if report_paths:
                print(f"  {report_paths.get('technical_md')}\n
{report_paths.get('customer_md')}\n  {report_paths.get('json')}")
            if summary["n_blockers"]:
                print(f"\n  GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
resolved.")
                sys.exit(2)
            print("\n  GATE: PASS ? review the flagged rallies, then `import-local`.")

        def curate(self):
            """Interactive loop to review and curate staging video annotations."""
            staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
            if not staging_videos:
                print("\nNo staging video annotations to review.\n")
                return

            print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
            table_data = []
            for idx, video in enumerate(staging_videos):
                table_data.append([
                    idx + 1,
                    video.video_id,
                    video.match_name,
```

```
                    video.sport.value,
                    video.license_type.value,
                    "YES" if video.is_commercial else "NO"
                ])

            print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
"License", "Commercial Safe?"], tablefmt="grid"))
            print("\nEnter a number (1-{}) to review details, or 'q' to
quit:".format(len(staging_videos)))

            choice = input("> ").strip()
            if choice.lower() == 'q':
                return

            try:
                idx = int(choice) - 1
                if idx < 0 or idx >= len(staging_videos):
                    print("Invalid choice.")
                    return
                self._curate_video_interactive(staging_videos[idx])
            except ValueError:
                print("Invalid input.")

        def _curate_video_interactive(self, video: VideoMetadata):
            """Displays details of a specific staging video and prompts for curation
action."""

            # Load and count events
            rallies = []
            pts = []
            delivs = []
            if video.sport == SportType.BADMINTON:
                rallies = self.db.get_badminton_rallies(video.video_id)
            elif video.sport == SportType.PICKLEBALL:
                rallies = self.db.get_pickleball_rallies(video.video_id)
            elif video.sport == SportType.TABLE_TENNIS:
                rallies = self.db.get_tabletennis_rallies(video.video_id)
            elif video.sport == SportType.TENNIS:
                pts = self.db.get_tennis_points(video.video_id)
            elif video.sport == SportType.CRICKET:
                delivs = self.db.get_cricket_deliveries(video.video_id)

            while True:
                print("\n================ REVIEWING VIDEO ================")
                print(f"Match Name   : {video.match_name}")
                print(f"Video ID     : {video.video_id}")
                print(f"Sport        : {video.sport.value}")
                print(f"Video URL    : {video.video_url or 'N/A'}")
                print(f"Local Path   : {video.local_path or 'N/A'}")
                print(f"License      : {video.license_type.value} ({video.license_url or 'no
URL'})")
                print(f"Commercial OK: {video.is_commercial}")

                if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
                    print(f"Total Rallies: {len(rallies)}")
                    if rallies:
                        print("Sample rallies:")
                        sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
                        print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
"Shots"], tablefmt="simple"))
                elif video.sport == SportType.TENNIS:
                    print(f"Total Points : {len(pts)}")
                    if pts:
```

```
730                         print("Sample points:")
731                         sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
732                         print(tabulate(sample_data, headers=["Point #", "Duration",
"Score"], tablefmt="simple"))
733                 elif video.sport == SportType.CRICKET:
734                     print(f"Total Balls  : {len(delivs)}")
735                     if delivs:
736                         print("Sample deliveries:")
737                         sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
738                         print(tabulate(sample_data, headers=["Over.Ball", "Duration",
"Runs", "Wicket"], tablefmt="simple"))
739
740                 print("\nChoose Curation Action:")
741                 print("  [a] Approve (promote to CURATED)")
742                 print("  [r] Reject (delete from database)")
743                 print("  [p] Play a rally/point segment in browser")
744                 print("  [k] Keep in Staging (do nothing and go back)")
745
746                 action = input("> ").strip().lower()
747                 if action == 'a':
748                     self.db.update_status(video.video_id, CurationStatus.CURATED)
749                     print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
750                     break
751                 elif action == 'r':
752                     self.db.delete_video(video.video_id)
753                     print(f"Successfully rejected and deleted '{video.match_name}'.\n")
754                     break
755                 elif action == 'k':
756                     print("Curation state unchanged.\n")
757                     break
758                 elif action == 'p':
759                     self._play_segment_interactive(video, rallies, pts, delivs)
760                 else:
761                     print("Invalid action selected.")
762
763         def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
764             import webbrowser
765
766             # Get start/end based on user input
767             start_time = None
768             end_time = None
769             label = "segment"
770
771             if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
772                 if not rallies:
773                     print("No rallies to play.")
774                     return
775                 idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
776                 try:
777                     rally_num = int(idx_str)
778                     rally = next((r for r in rallies if r.rally_number == rally_num), None)
779                     if rally:
780                         start_time = rally.start_time
781                         end_time = rally.end_time
782                         label = f"Rally {rally_num}"
783                 except ValueError:
784                     pass
785             elif video.sport == SportType.TENNIS:
786                 if not pts:
787                     print("No points to play.")
788                     return
```

```
                idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
                try:
                    pt_num = int(idx_str)
                    pt = next((p for p in pts if p.point_number == pt_num), None)
                    if pt:
                        start_time = pt.start_time
                        end_time = pt.end_time
                        label = f"Point {pt_num}"
                except ValueError:
                    pass
            elif video.sport == SportType.CRICKET:
                if not delivs:
                    print("No deliveries to play.")
                    return
                over_str = input("Enter Over number: ").strip()
                ball_str = input("Enter Ball number: ").strip()
                try:
                    o_num = int(over_str)
                    b_num = int(ball_str)
                    d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
None)
                    if d:
                        start_time = d.start_time
                        end_time = d.end_time
                        label = f"Over {o_num} Ball {b_num}"
                except ValueError:
                    pass

            if start_time is None or end_time is None:
                print("Segment not found or invalid input.")
                return

            print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")

            # Check video location
            play_url = None
            if video.local_path and os.path.exists(video.local_path):
                abs_path = os.path.abspath(video.local_path)
                # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
                play_url = f"file:///{abs_path.replace(os.sep,
'/')}#t={start_time},{end_time}"
            elif video.video_url:
                # YouTube URL: seek to start time
                if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
                    t_sec = int(start_time)
                    sep = "&" if "?" in video.video_url else "?"
                    play_url = f"{video.video_url}{sep}t={t_sec}s"
                else:
                    play_url = video.video_url

            if play_url:
                print(f"Opening browser: {play_url}")
                webbrowser.open(play_url)
            else:
                print("Error: No local video file or remote URL available to play.")


    def main():
        parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
        subparsers = parser.add_subparsers(dest="command")

        # Status command
        subparsers.add_parser("status", help="Display summary status of sports metadata
database")
```

```
850
851         # Curate command
852         subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")
853
854         # Import-local command
855         import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
856         import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
857         import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
858         import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
859         import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
860         import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
861         import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
862         import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
863         import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
864
865         # Fetch command ? (re)download videos at best resolution, update DB in place
866         fetch_parser = subparsers.add_parser(
867             "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
868         fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
869         fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
870         fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
871                                   help="Which curation statuses to fetch (default: all)")
872         fetch_parser.add_argument("--min-height", type=int, default=720,
873                                   help="Skip videos already at/above this height unless
--force (default: 720)")
874         fetch_parser.add_argument("--max-height", type=int, default=None,
875                                   help="Cap download resolution (e.g. 1080); default: best
available")
876         fetch_parser.add_argument("--force", action="store_true",
877                                   help="Re-download even if a good-enough local file already
exists")
878         fetch_parser.add_argument("--include-noncommercial", action="store_true",
879                                   help="Include videos whose license is not commercially
safe (default: commercial only)")
880         fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
881                                   help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
882                                        "A Premium login gives reliable best-resolution
downloads. "
883                                        "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
884         fetch_parser.add_argument("--cookies", metavar="FILE",
885                                   help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
886         fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
887                                   help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
888                                        "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
889         fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
890                                   help="Pause between videos to avoid YouTube rate-limiting
```

```
(default: 4)")
        fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
                                  help="Retries per video when a download fails or returns a
low format (default: 2)")
        fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
                                  help="Base backoff between retries; grows linearly per
attempt (default: 20)")

        # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
        cvat_parser = subparsers.add_parser(
            "convert-cvat",
            help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
        cvat_parser.add_argument("--input", required=True,
                                 help="Path to the CVAT XML export (image- or track-mode)")
        cvat_parser.add_argument("--out", required=True,
                                 help="Path to write the canonical per-rally CSV (import-
local input)")
        cvat_parser.add_argument("--video-path",
                                 help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
        cvat_parser.add_argument("--fps", type=float,
                                 help="Override fps instead of probing the video")
        cvat_parser.add_argument("--step", type=int,
                                 help="Frame sampling step (default: read from CVAT meta,
else inferred)")
        cvat_parser.add_argument("--issues",
                                 help="Path to also write the human-readable issues report")

        # Validate-delivery command ? run the full check suite + emit a structured report
        vd_parser = subparsers.add_parser(
            "validate-delivery",
            help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
        vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
        vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
        vd_parser.add_argument("--video-path",
                               help="Source video; fps via ffprobe (when using --input)")
        vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
        vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
        vd_parser.add_argument("--out-dir", default="data/validation",
                               help="Where to write the report + review sheet")

        # Export command
        export_parser = subparsers.add_parser(
            "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
        export_parser.add_argument("--out-dir", default="data/eval_export",
                                   help="Output directory for per-video CSVs and
manifest.json")
        export_parser.add_argument("--sport",
                                   help="Restrict to a single sport (default: all
supported_sports)")
        export_parser.add_argument("--include-staging", action="store_true",
                                   help="Also export videos still in staging (default:
curated only)")
        export_parser.add_argument("--include-noncommercial", action="store_true",
                                   help="Include videos whose license is not commercially
safe (default: commercial only)")

```

```
938         args = parser.parse_args()
939         cli = SportsDataCLI()
940 
941         if args.command == "status":
942             cli.status()
943         elif args.command == "curate":
944             cli.curate()
945         elif args.command == "import-local":
946             cli.import_local(args)
947         elif args.command == "convert-cvat":
948             cli.convert_cvat(args)
949         elif args.command == "validate-delivery":
950             cli.validate_delivery(args)
951         elif args.command == "export":
952             cli.export_eval_set(args)
953         elif args.command == "fetch":
954             cli.fetch(args)
955         else:
956             # Default: print help
957             parser.print_help()
958 
959     if __name__ == "__main__":
960         main()
961 
```

5. user (2026-06-10T06:10:06.798Z)

```
C:\Users\avidu\Projects\sports-data-collector\README.md:90:[Omitted long matching line]
C:\Users\avidu\Projects\sports-data-collector\docs\PIPELINE_ROLE.md:19:The strategic goal is to
replace that with an **offline model trained on our own
C:\Users\avidu\Projects\sports-data-
collector\docs\MANUAL_INGESTION_POSITIONING.md:55:`src/cli/normalize_annotations.py` pre-import
normalizer fixes both, as a **standalone
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ANNOTATION_GUIDE.md:7:>
**TODO(avidullu): replace every `[SCREENSHOT: ...]` placeholder with a real frame grab
```

6. user (2026-06-10T06:10:10.905Z)

```
C:\Users\avidu\Projects\sports-data-collector\tests\test_cli.py:192:
cli.db.insert_badminton_rallies([
C:\Users\avidu\Projects\sports-data-collector\tests\test_db.py:54:    inserted =
temp_db.insert_badminton_rallies(rallies)
C:\Users\avidu\Projects\sports-data-collector\tests\test_db.py:72:
temp_db.insert_badminton_rallies([
C:\Users\avidu\Projects\sports-data-collector\tests\test_db.py:95:
temp_db.insert_badminton_rallies([
```

7. user (2026-06-10T06:10:17.962Z)

```
75      To register a local video file alongside its JSON/CSV annotation file (bypasses
staging):
76      ```bash
77      python -m src.cli.curate import-local --sport badminton --video-path
"data/videos/my_custom_match.mp4" --annotations-path "data/mock_local_badminton_anno.json"
--match-name "My Custom Local Match" --license Proprietary
78      ```
79      `import-local` works for every configured sport ? `badminton`, `tennis`, `cricket`,
`pickleball`, and `tabletennis` ? from either a JSON or a CSV annotations file. For the
canonical rally CSV schema and the rater-facing instructions used to produce these files (e.g.
via an annotation vendor), see [`docs/annotation/`](docs/annotation/).
80
81      ### 5. Fetch / Upgrade Videos to Best Resolution
82      Videos are **always downloaded at the best resolution available** (via yt-dlp `bv*+ba/b
-S res,...`, merged to MP4); the actual width/height/fps is ffprobed and stored ? never assumed.
```

```
The `fetch` command (re)downloads videos for entries already in the DB and updates
`local_path`/`resolution`/`fps` **in place**, preserving all rally annotations and curation
status (no clean slate needed):
83      ```bash
84      # Upgrade every sub-720p video to best resolution, using a logged-in browser's cookies:
85      python -m src.cli.curate fetch --cookies-from-browser firefox
86
87      # A single video, forcing a re-download even if a good local copy exists:
88      python -m src.cli.curate fetch --video-id shuttleset_1 --force --cookies-from-browser
firefox
89      ```
90      Flags: `--sport`, `--video-id`, `--status {staging,curated,all}` (default `all`),
`--min-height` (skip videos already at/above it unless `--force`; default `720`), `--max-height`
(cap resolution, e.g. `1080`; default best available), `--force`, `--include-noncommercial`,
`--cookies-from-browser`/`--cookies`, `--player-client`. A global resolution cap can also be set
via `video_storage.max_height` in `config/config.yaml`. Re-runs are cheap and resumable; a
download that falls below `--min-height` is treated as a **failure** (discarded, DB left
unchanged) so a flaky low-res result never overwrites a good entry.
91
92      > **Authenticated cookies are strongly recommended.** Anonymous YouTube downloads are
throttled and non-deterministic (often falling back to 144p). Pass `--cookies-from-browser
firefox` (or `chrome`/`edge`/`brave`, fully closed) ? or `--cookies cookies.txt` ? to use a
logged-in (ideally Premium) session for fast, reliable best-resolution downloads. Chrome's app-
bound cookie encryption may block direct reads on Windows; Firefox reads cleanly even while
open.
93      >
94      > **JS runtime:** yt-dlp needs one (e.g. [deno](https://deno.com)) to solve YouTube's
challenges for high-res formats; `fetch` auto-detects a deno install at `~/.deno/bin`. The
default `--player-client` avoids `web`/`web_safari`, which YouTube forces into SABR streaming
(drops to 144p).
95
96      ### 6. Export Annotations as Indexer-Ready Eval/Training Sets
97      To turn curated annotations into ground-truth files the badminton-highlight-indexer can
score and train against, export per-video `start,end` CSVs plus a `manifest.json`:
98      ```bash
99      # Curated + commercially-safe videos across all supported sports (default):
100     python -m src.cli.curate export --out-dir data/eval_export
101
102     # One sport, also including staging entries:
103     python -m src.cli.curate export --sport badminton --include-staging
104     ```
105     The exporter writes `<out-dir>/<sport>/<video_id>.csv` (decimal-second `start,end` rows)
and a `manifest.json` pairing each CSV with its video `local_path`/`video_url`, fps, resolution,
and license. The indexer keys videos by **file MD5**, not by our `video_id`, so the downstream
eval/training run must process the exact video file named in the manifest. Flags: `--sport`
(restrict to one sport), `--include-staging` (default: curated only), `--include-noncommercial`
(default: commercial-safe only).
106
107     ---
108
109     ## Configuration
```

8. user (2026-06-10T06:10:18.781Z)

```
40          video = VideoMetadata(
41              video_id="match_2",
42              sport=SportType.BADMINTON,
43              license_type=LicenseType.MIT,
44              is_commercial=True,
45              match_name="Match 2"
46          )
47          temp_db.insert_video(video)
48
49          rallies = [
50              BadmintonRally(video_id="match_2", rally_number=1, start_time=10.0,
```

```
end_time=25.0, score_before="0-0", score_after="1-0", shots_count=5),
51              BadmintonRally(video_id="match_2", rally_number=2, start_time=30.0,
end_time=45.0, score_before="1-0", score_after="1-1", shots_count=12)
52          ]
53
54          inserted = temp_db.insert_badminton_rallies(rallies)
55          assert inserted == 2
56
57          fetched_rallies = temp_db.get_badminton_rallies("match_2")
58          assert len(fetched_rallies) == 2
59          assert fetched_rallies[0].rally_number == 1
60          assert fetched_rallies[1].shots_count == 12
61
62      def test_reinsert_video_preserves_child_rallies(temp_db):
63          """Re-inserting an existing video must update it in place, not wipe its rallies (bug
3)."""
64          video = VideoMetadata(
65              video_id="match_upsert",
66              sport=SportType.BADMINTON,
67              license_type=LicenseType.MIT,
68              is_commercial=True,
69              match_name="Original Name",
70          )
71          temp_db.insert_video(video)
72          temp_db.insert_badminton_rallies([
73              BadmintonRally(video_id="match_upsert", rally_number=1, start_time=1.0,
end_time=5.0),
74          ])
75
76          # Re-insert the same video_id with a changed field.
77          video.match_name = "Updated Name"
78          temp_db.insert_video(video)
79
80          fetched = temp_db.get_video("match_upsert")
81          assert fetched.match_name == "Updated Name"
82          # Child rallies must still be present.
83          assert len(temp_db.get_badminton_rallies("match_upsert")) == 1
84
85
86      def test_get_segment_intervals_badminton(temp_db):
87          """The sport-agnostic interval getter returns chronologically-ordered
88          (start, end) tuples ? the shape the eval/training exporter consumes."""
89          video = VideoMetadata(
90              video_id="match_iv", sport=SportType.BADMINTON,
91              license_type=LicenseType.MIT, is_commercial=True, match_name="Match IV",
92          )
93          temp_db.insert_video(video)
94          # Insert out of order to prove ORDER BY rally_number is applied.
95          temp_db.insert_badminton_rallies([
96              BadmintonRally(video_id="match_iv", rally_number=2, start_time=30.0,
end_time=45.0),
97              BadmintonRally(video_id="match_iv", rally_number=1, start_time=10.0,
end_time=25.0),
98          ])
99
100         intervals = temp_db.get_segment_intervals("match_iv", SportType.BADMINTON)
101         assert intervals == [(10.0, 25.0), (30.0, 45.0)]
102
103
104     def test_pickleball_rally_insertion(temp_db):
105         video = VideoMetadata(
106             video_id="pb_1",
107             sport=SportType.PICKLEBALL,
108             license_type=LicenseType.MIT,
109             is_commercial=True,
```

```
110             match_name="Pickleball Match 1",
111         )
112         temp_db.insert_video(video)
113
114         rallies = [
115             PickleballRally(video_id="pb_1", rally_number=1, start_time=10.0, end_time=25.0,
shots_count=6, ending_reason="forced_error", last_shot_outcome="out"),
116             PickleballRally(video_id="pb_1", rally_number=2, start_time=30.0, end_time=33.0,
shots_count=1, ending_reason="service_fault"),
117         ]
118         assert temp_db.insert_pickleball_rallies(rallies) == 2
119
```

9. user (2026-06-10T06:10:33.743Z)

```
C:\Users\avidu\Projects\sports-data-collector\docs\DECISIONS.md:71:5. `config/config.yaml` ? add
the sport to `supported_sports` (and confirm the `SportType` enum value in `schemas/base.py`).
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:13:Position
the collector as the **single canonical system-of-record for golden rally
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:108:  a
*workflow* state, not a *trust* state. Add a separate `needs_review ? confirmed ?
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ROORA_BRIEF.md:131:7. Confirmation
you can deliver the CSV schema above.
C:\Users\avidu\Projects\sports-data-
collector\docs\annotation\NDA_TEMPLATE.md:54:**TODO(avidullu): confirm with counsel whether a
separate data-processing addendum is needed.**
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\NDA_TEMPLATE.md:69:the property of
**TODO(avidullu): confirm IP ownership ? typically Party A** (to be confirmed in
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\NDA_TEMPLATE.md:78:e.g. Bengaluru,
Karnataka** have exclusive jurisdiction. **TODO(avidullu): confirm with counsel.**
```

10. user (2026-06-10T06:10:37.539Z)

```
data/mock_local_badminton_anno.json
data/roora_badminton_canonical.csv
data/roora_badminton_issues.txt
data/sports_metadata.db
data/staging_coachai_match.csv
data/validation/pilot_badminton_01_report.json
data/validation/pilot_badminton_01_report.md
data/validation/pilot_badminton_01_review.csv
---
```

---- Python ----

```
__pycache__/
*.py[cod]
*$py.class
*.egg-info/
.eggs/
build/
dist/
.installed.cfg
*.egg
```

---- Test / coverage ----

```
.pytest_cache/
.coverage
.coverage.*
htmlcov/
.tox/
.cache/
```

---- Virtual environments ----

```
.venv/
venv/
env/
ENV/
```

---- Secrets / local config ----

```
.env
.env.*
```

---- OS / editor cruft ----

```
.DS_Store
Thumbs.db
.idea/
.vscode/
*.swp
```

---- Project: large media kept outside git (see README) ----
The video corpus is hundreds of MB+; only metadata (the SQLite DB) and the small source annotation CSVs are version-controlled. Re-fetch videos from the stored video_url / local_path per the manifest.

```
data/videos/
```

---- Generated eval/training exports (regenerated via `curate export`) ----

```
data/eval_export*/
```

---- Generated temp artifacts (crawler scratch space; regenerated each run) ----

```
data/temp/
```

---- Per-video observability reports (written next to videos / inputs at runtime) ----

```
*.report.json
*.report.md
*.summary.md
```

11. user (2026-06-10T06:10:37.676Z)

```
68:- Add a `split: "train" | "eval"` field to the collector's `curate export`
79:- Requires a collector-side manifest change (coordinate the cross-repo contract).
```

12. user (2026-06-10T06:10:54.419Z)

```
1       # Positioning: manual (rally-annotator) golden-label ingestion
2
3       How `sports-data-collector` should be positioned to ingest **manual** rally labels
4       produced by the VLC [rally-annotator](https://github.com/avidullu/rally-annotator),
5       alongside vendor (Roora/CVAT) deliveries, and serve them to the
6       badminton-highlight-indexer eval/training pipeline.
7
8       > Source: a 10-agent analysis workflow across this repo, the indexer, and the
9       > annotator, verified against the real `Adarsh and Avi.rallies.csv`. Confidence: high.
10
11      ## Bottom line
12
13      Position the collector as the **single canonical system-of-record for golden rally
14      labels**: the one place where *any* annotation source ? manual VLC lean CSV, the rich
15      review sheet, or vendor CVAT/CSV ? becomes a validated, license-gated, registered rally
16      record. The indexer consumes **only** the collector's narrow export contract:
17      `manifest.json` + a per-video `start,end` CSV (`curate.py::export_eval_set`;
18      `calibrate_wasb.py::load_golden_gts` reads `start_time`/`end_time` only).
```

The load-bearing property to protect: **the seam to the indexer stays intervals-only.** Everything else ? `ending_reason`, `shots_count`, scores, provenance, maturity, the labeled window ? is validated at the hub and rides in the **manifest**, never the eval CSV. A new sport or source becomes an adapter change at the hub, never an indexer change.

Role in the pipeline

```
 producers ???????????????  collector (hub) ?????????????  consumer
 ? VLC rally-annotator       parse ? validate ? license-gate    badminton-highlight-
   (lean 5-col CSV)          ? register (SQLite) ? export        indexer (eval/training)
 ? manual review sheet                                          reads ONLY:
   (rich 14-col CSV)         everything but (start,end) is        manifest.json +
 ? vendor Roora / CVAT       validated then dropped before        per-video start,end CSV
   (CVAT XML / canonical)    the indexer sees it
```

It already works today (zero code changes)

The lean annotator CSV header `rally_number,start_time,end_time,ending_reason,sport` matches the `import-local` `DictReader` exactly (`curate.py::_parse_local_annotations`), and the validator accepts gapped/partial coverage as-is (`validator.py::validate_badminton_rallies` checks only positive duration + no overlap). Verified end-to-end on the real file ? 40 rallies ? export ? 40-interval indexer CSV.

```bash
python -m src.cli.curate import-local --sport badminton \
  --video-path "?/Adarsh and Avi.MP4" \
  --annotations-path "?/Adarsh and Avi.rallies.csv" --license Proprietary --fps 59.94
python -m src.cli.curate export   # ? manifest.json + <video_id>.csv (start,end)
```

P0 ? the two-landmine guard (`normalize_annotations.py`, shipped)

Partial human data has two failure modes the raw path handles badly. The `src/cli/normalize_annotations.py` pre-import normalizer fixes both, as a **standalone step** that feeds `import-local` (it does **not** touch the DB schema or the `rally_number`-ordered export):

1. **An unusable time row aborts the *entire* import.** A blank/missing time makes `safe_float_required` raise (`parsing.py`); a reversed `end_time <= start_time` or a negative `start_time` makes the `BadmintonRally` pydantic validator raise (`schemas/badminton.py::validate_timestamps`). Either way the `try/except` in `import-local` turns it into `sys.exit(1)`, killing the whole video. ? the normalizer **drops** such rows (mirroring the indexer loader, which already skips `not end > start`).
2. **A fractional/duplicate `rally_number` (e.g. `7.5`, `16.5`) silently corrupts data.** `int(float("7.5")) ? 7` (`parsing.py::safe_int`) collides on the `(video_id, rally_number)` primary key, and the DELETE-then-INSERT upsert (`db.py::insert_badminton_rallies`) destroys the colliding rally. ? the normalizer **renumbers `1..N` by `start_time`**, recording the original id for traceability.

It also converts `mm:ss`/`HH:MM:SS` ? decimal seconds and folds a pipe-delimited `ending_reason` ("forced_error|other") to its primary token.

```bash
python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
python -m src.cli.curate import-local --sport badminton --video-path "?/Match.MP4" \
  --annotations-path clean.csv --license Proprietary
```

```
80      **Do not widen the PK to float** ? the DB column is INTEGER, export orders by it, and
81      the indexer ignores `rally_number` entirely. Resolve ids at the adapter boundary.
82
83      ## Annotator ? canonical field mapping
84
85      | Annotator field | Canonical | Handling |
86      |---|---|---|
87      | `rally_number` (lean) / `rally` (rich) | `rally_number:int` | Lean maps directly;
**fractional/dup ids renumbered at the normalizer** (PK is INT) |
88      | `start_time` / `start_mmss` | `start_time:float` | Decimal direct; `mm:ss` converted
to seconds |
89      | `end_time` / `end_mmss` | `end_time:float` | **Blank / ? start dropped, not raised** |
90      | `ending_reason` | `ending_reason:str?` | Vocab matches the documented canonical set
(free-text, not an enforced enum); pipe-delimited folded to primary; **stored but dropped on
export** |
91      | `sport` | ? (from `--sport` flag) | CSV column ignored; operator sets the flag |
92      | rich-only `shots`, `true_*`, `VERIFIED`, `notes` | `shots_count` + QA/provenance |
Carried where present; none reach the indexer |
93
94      ## Roadmap to the canonical-hub end-state
95
96      All additive, video-level, backward-compatible (consumers ignore unknown manifest keys).
97
98      - **P1 ? persist the labeled window in the manifest.** Partial labels cover only a
99        prefix; a full-video eval penalizes correct detections in the unlabeled tail. The
100       indexer already restricts to a window (`eval_partial_labels.py::restrict_to_window`)
101       but currently *infers* `window_end = max(end_time)` ? which is **wrong when a video is
102       tagged past its last rally** (valid predictions in the labeled-but-rally-free tail get
103       scored as false positives). The collector should own `labeled_window_start/end`
104       (the VLC tool is resumable + monotonic, so the session extent is known), carry it in
105       the manifest, and let the indexer apply it automatically. Short-term: pass
106       `--window-end` at eval time.
107     - **P1 ? maturity ladder + provenance.** `CurationStatus` (STAGING/CURATED/REJECTED) is
108       a *workflow* state, not a *trust* state. Add a separate `needs_review ? confirmed ?
109       gold` enum + `annotation_source` (manual_vlc / manual_review / vendor_cvat) +
110       `annotator_id` + timestamp. Route self-rated imports to `needs_review` instead of the
111       hardcoded `CURATED` (`curate.py`), so the golden-set regression flow restricts the
gold
112       tier to `maturity=gold` and an unreviewed re-tag can't silently become the answer key.
113       (Today a single-rater self-rating and a vendor-QA'd delivery are indistinguishable.)
114     - **P2 ? unify sources behind one adapter** emitting `(List[RallyRecord], List[Issue])`,
115       and run the **same Issue detectors** (gap>15s possible-missed-rally, shot-density,
116       vocab, overlap) on manual imports that vendor CVAT already gets. Today self-produced
117       labels get *less* scrutiny than a vendor delivery ? backwards.
118     - **P2 ? freeze the intervals-only export contract** in one doc both repos reference +
119       a round-trip test (header matching is exact case-insensitive, so a vendor
120       `start_seconds` would fail silently).
121
122     ## Open question
123
124     Is the **rich** `rally_review.csv` (mm:ss times, a `rally` column with fractional ids,
125     pipe-delimited reasons, `true_*` correction columns) ever the *source of truth*, or is
126     the lean VLC `.rallies.csv` always canonical? Neither `import-local` nor
127     `load_canonical_csv` (which keys on `rally_number`) ingest the rich file directly. The
128     `normalize_annotations.py` step already handles its `mm:ss` + `rally` + pipe-reason
129     quirks for the common case; a fuller converter that applies `true_*` overrides is only
130     worth building if the rich sheet is canonical.
131
```

13. assistant (2026-06-10T06:11:36.780Z)

The positioning doc is important context. Let me do a final check for any backup/guard logic anywhere in the collector source.

14. user (2026-06-10T06:11:38.644Z)

```
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py:877:
help="Re-download even if a good-enough local file already exists")
C:\Users\avidu\Projects\sports-data-collector\src\reporting\adapter.py:3:Reuses the existing
rally_cvat report primitives (``Issue``, records, severities)
C:\Users\avidu\Projects\sports-data-collector\src\core\db.py:169:        """Insert a video
entry, or update it in place if it already exists.
C:\Users\avidu\Projects\sports-data-collector\src\core\db.py:215:            # First clear
existing rallies for this video_id to avoid key conflicts on updates
```

15. user (2026-06-10T06:12:04.024Z)

```
Structured output provided successfully
```

16. assistant (2026-06-10T06:12:09.296Z)

```
Verification complete.
```